DECK 3 OF 3

SECURITY MONITORING

Wazuh | Snort | Zeek. Detect. Analyze. Respond.

Wazuh (host) | Snort (network IDS/IPS) | Zeek (network recorder)

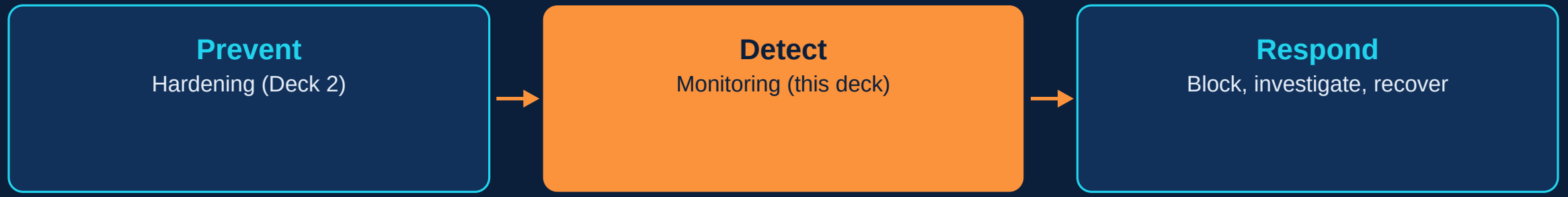
Presented by: Tyrone Justine A. Medina

OVERVIEW

Agenda

- **Part 1: Why monitoring**
  Prevent, detect, respond. SIEM, IDS, IPS and NSM explained
- **Part 2: Wazuh**
  Host-based SIEM and XDR: install, agents, FIM, rules, active response
- **Part 3: Snort**
  Signature-based network IDS/IPS: modes, config, rules, commands
- **Part 4: Zeek**
  Network security monitor: logs, deployment, analysis, scripts
- **Part 5: Putting it together**
  Comparison, layered setup, alert triage workflow

Why Monitoring?



- **Assume breach**
  no prevention is perfect
- **Dwell time**
  the sooner you detect, the smaller the damage
- Without logs and network records you cannot answer "what happened?"

Four Terms You Must Know

SIEM

Collects and analyzes logs from many sources, correlates events and raises alerts.

IDS

Intrusion Detection System. Detects and alerts, does not block.

IPS

Intrusion Prevention System. Detects and blocks, sits inline.

NSM

Network Security Monitoring. Records network activity for investigation.

Host-based tools watch what happens inside a server (Wazuh). Network-based tools watch the traffic between devices (Snort, Zeek).

What Do We Actually Watch?

Source	What it tells you	Tool
/var/log/auth.log	Logins, sudo, SSH failures	Wazuh, Fail2ban
Files in /etc and /usr/bin	Tampering, persistence	Wazuh FIM
Network packets	Scans, exploits, known malware traffic	Snort
Connection metadata	Who talked to whom, DNS, TLS	Zeek
Installed package versions	Known vulnerabilities (CVE)	Wazuh vulnerability detection
Configuration state	Drift from a CIS baseline	Wazuh SCA

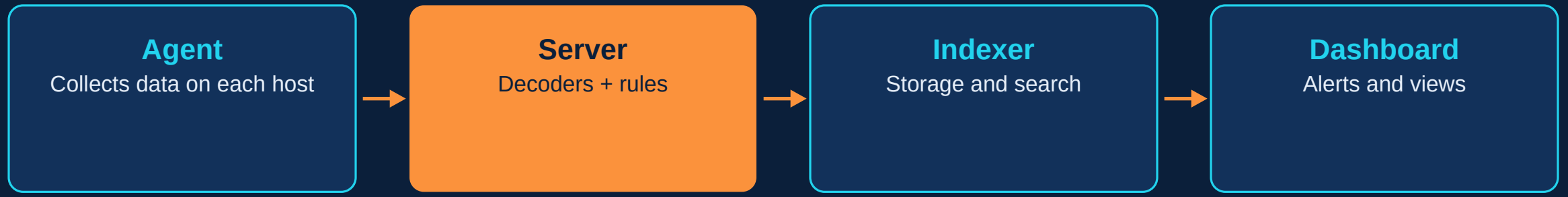
Wazuh: Overview

- Open-source security platform: SIEM + XDR
- Host-based: lightweight agent on each endpoint
- Supports Linux, Windows, macOS, cloud and containers
- Forked from OSSEC, maps detections to MITRE ATT&CK
- Free, with a large community and thousands of built-in rules

Quick facts

Agents: on every endpoint
Rules: thousands built in
Platforms: Linux / Win / Mac / Cloud
License: Open Source

Wazuh: Architecture



- Agents send events to the server over an encrypted channel
- The server decodes each log line, then matches it against rules
- The indexer stores alerts, the dashboard is where analysts work
- A small lab can run all four components on one machine (all-in-one)

Wazuh: Capabilities

Log Analysis

Rules and decoders turn raw log lines into meaningful alerts.

File Integrity

Detects changes in /etc, binaries and web roots.

Vulnerabilities

Matches installed packages against known CVEs.

SCA

Security Configuration Assessment against CIS benchmarks.

Active Response

Automatically blocks an IP or runs a script.

Compliance

Maps alerts to PCI DSS, GDPR, HIPAA and more.

Wazuh: Ports to Know

Port	Protocol	Used for
1514	TCP	Agents sending events to the server
1515	TCP	Agent enrollment (registration)
55000	TCP	Wazuh server API
9200	TCP	Wazuh indexer API
443	TCP	Dashboard (HTTPS)

> *Open 1514 and 1515 only from your agent networks, and keep 9200 and 55000 internal.*

Wazuh: Server Installation

INSTALL

```
# Server: all-in-one install
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh
sudo bash ./wazuh-install.sh -a

# Read the generated admin password
sudo tar -O -xvf wazuh-install-files.tar \
  wazuh-install-files/wazuh-passwords.txt

# Check the services
sudo systemctl status wazuh-manager
sudo systemctl status wazuh-indexer wazuh-dashboard

# Dashboard: https://<server-ip>   user: admin
```

- **Lab sizing**
  roughly 4 vCPU and 8 GB RAM for a small all-in-one
- **Version**
  the 4.14 in the URL changes, check the Wazuh docs for the latest
- Change default passwords and keep the install files safe

Wazuh: Agents and Manager Commands

ENDPOINT (AGENT)

```
# After adding the Wazuh repo
sudo WAZUH_MANAGER="10.0.0.5" \
  apt install wazuh-agent
sudo systemctl daemon-reload
sudo systemctl enable --now wazuh-agent

# Agent log
sudo tail -f /var/ossec/logs/ossec.log
```

MANAGER

```
# List connected agents
sudo /var/ossec/bin/agent_control -l

# Test how a log line is decoded
sudo /var/ossec/bin/wazuh-logtest

# Watch alerts live
sudo tail -f /var/ossec/logs/alerts/alerts.json

# Service status
sudo /var/ossec/bin/wazuh-control status
```

> *Use wazuh-logtest every time you write or change a rule. Paste a real log line and see which rule fires.*

Wazuh: File Integrity Monitoring

FIM

```
<!-- ossec.conf on the agent -->
<syscheck>
  <frequency>43200</frequency>
  <directories check_all="yes" realtime="yes">
    /etc,/usr/bin
  </directories>
  <directories check_all="yes" report_changes="yes">
    /var/www
  </directories>
  <ignore>/etc/mtab</ignore>
</syscheck>

sudo systemctl restart wazuh-agent
```

- **realtime**
  alert immediately instead of waiting for the next scan
- **report_changes**
  shows what changed inside text files
- **Use case**
  an alert if sshd_config changes after a brute-force attempt
- Monitor important paths only, noisy paths create alert fatigue

Wazuh: A Custom Rule

CUSTOM RULE

```
<!-- /var/ossec/etc/rules/local_rules.xml -->
<group name="local,syslog,">
  <rule id="100100" level="10">
    <if_sid>5402</if_sid>
    <match>/etc/shadow</match>
    <description>Sudo used on /etc/shadow</description>
  </rule>
</group>

# Test with a sample log line, then apply
sudo /var/ossec/bin/wazuh-logtest
sudo systemctl restart wazuh-manager
```

- **Rule IDs**
  use 100000 and above for your own rules
- **if_sid**
  builds on another rule, here 5402 (successful sudo to root)
- **level**
  0 to 15, higher means more severe
- **Decoders**
  extract fields from raw logs before rules run

Wazuh: Active Response

ACTIVE RESPONSE

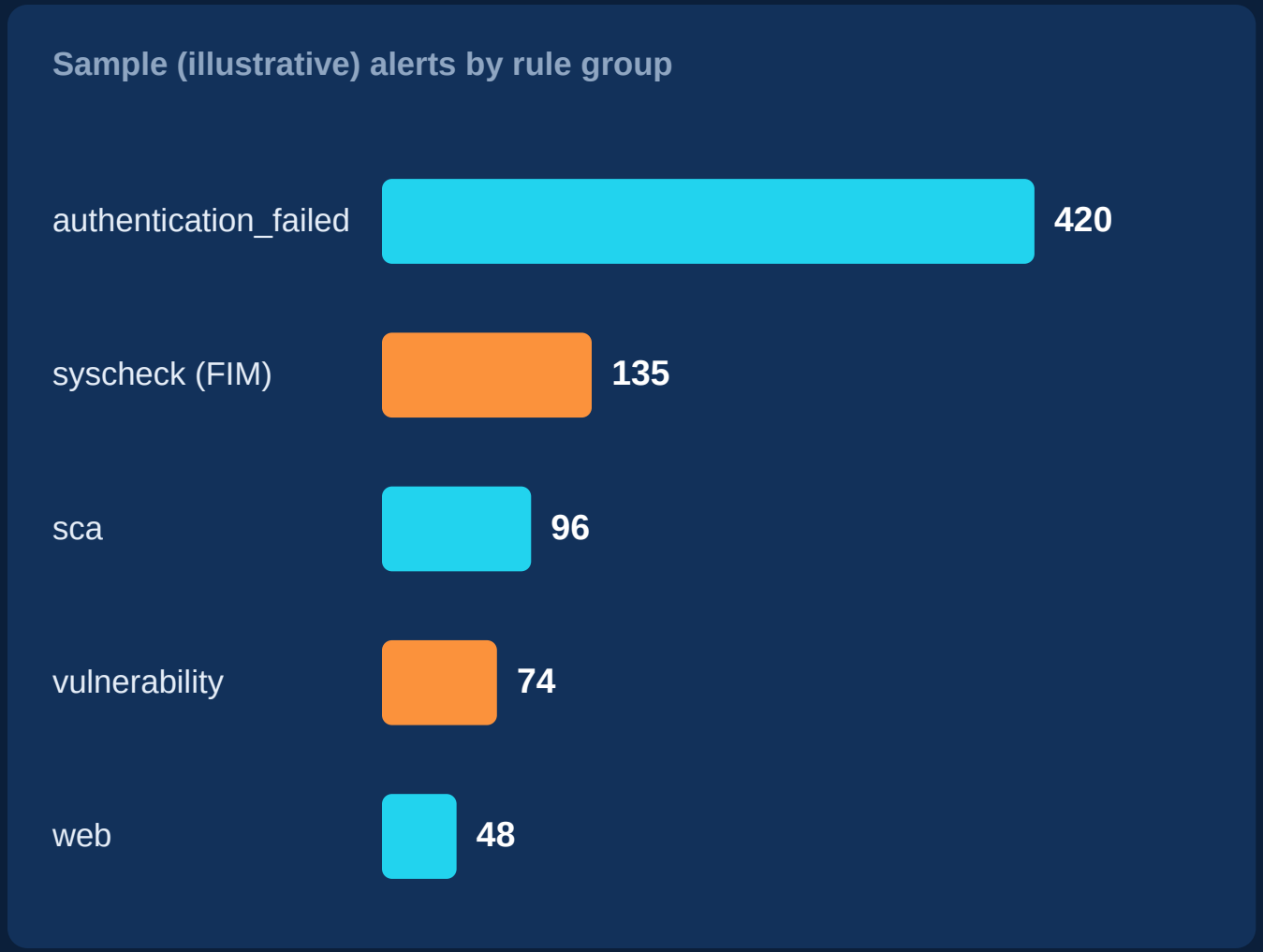
```
<!-- manager ossec.conf: auto-block SSH brute force -->
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>5712</rules_id>
  <timeout>600</timeout>
</active-response>

sudo systemctl restart wazuh-manager
```

- **firewall-drop**
  blocks the source IP with the host firewall
- **location local**
  runs on the agent that saw the attack
- **timeout 600**
  removes the block after 10 minutes
- Whitelist your admin IPs so you never block yourself

Wazuh Use Case: SSH Brute Force

- FIM alert if sshd_config changes afterward
- Integrates with Slack, email and TheHive
- Replaces manual Fail2ban + auth.log checks

Wazuh: Reading the Dashboard



- **authentication_failed**
  the biggest group, usually scanners and brute force
- **syscheck**
  file changes, review the unexpected ones
- **sca + vulnerability**
  your hardening to-do list
- Filter by agent, rule group or MITRE technique

Snort: Overview and Modes

- Network IDS/IPS created by Martin Roesch, now owned by Cisco
- Signature-based: compares packets to rules
- Rules come from Cisco Talos, the community, or your own
- Snort 3: multi-threaded, new rule syntax, Lua configuration

Three modes

Sniffer: shows packets on screen
Packet Logger: saves packets to disk
NIDS / NIPS: detection and prevention, the real use

Snort: Packet Pipeline

- Every packet goes through the same stages
- Capture uses the DAQ library (afpacket, pcap) to read from an interface
- Inspectors rebuild streams and understand protocols before rules run
- The rule action decides what happens: alert, drop, or just log

Snort: IDS vs IPS Deployment

IDS mode (passive)

- Connected to a switch mirror/SPAN port
- Watches a copy of the traffic
- Safe: cannot disrupt the network
- Alerts only, cannot block

IPS mode (inline)

- Traffic passes through Snort
- Can drop malicious packets
- Risk: false positives block real traffic
- Tune rules before enabling

> *Start in IDS mode. Review alerts for a few weeks, suppress noisy rules, then move only proven rules to drop.*

Snort: snort.lua Essentials

CONFIG

```
-- snort.lua (excerpt)
HOME_NET = '192.168.1.0/24'
EXTERNAL_NET = '!$HOME_NET'

ips =
{
    enable_builtin_rules = true,
    include = RULE_PATH .. '/local.rules',
    variables = default_variables
}

alert_fast = { file = true }

-- Validate from the shell:
-- snort -c /usr/local/etc/snort/snort.lua -T
```

- **HOME_NET**
  the network you protect, rules use it to know "inside" vs "outside"
- **ips**
  which rule files and built-in rules to load
- **alert_fast**
  one-line alerts written to the log folder
- Always validate with -T before running

Snort: Writing Rules

RULES

```
# local.rules
alert tcp any any -> $HOME_NET 22 (
  msg:"Possible SSH brute force";
  flags:S;
  detection_filter:track by_src, count 5, seconds 60;
  sid:1000001; rev:1;)

alert icmp any any -> $HOME_NET any (
  msg:"ICMP ping"; itype:8;
  sid:1000002; rev:1;)
```

- **Rule = header + options**
  Header: action proto src -> dst. Options in ( ).
- **sid**
  custom SIDs start at 1,000,000
- **rev**
  increase it each time you edit the rule
- **detection_filter**
  alert only after 5 SYNs from one source in 60 seconds

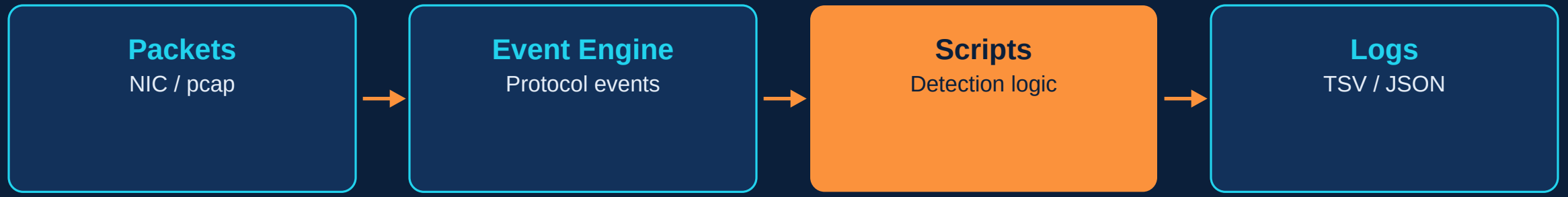
Snort: Commands

COMMANDS

```
# Validate config, then run (IDS)
snort -V
snort -c /usr/local/etc/snort/snort.lua -T
sudo snort -c snort.lua -R local.rules \
  -i eth0 -A alert_fast -k none

# Read a pcap
snort -c snort.lua -R local.rules \
  -r capture.pcap -A alert_fast

# Inline IPS
sudo snort -c snort.lua -R local.rules \
  --daq afpacket -i eth0:eth1 -Q
```

- **-T**
  test configuration and exit
- **-R**
  load a rules file
- **-r**
  read a saved capture, ideal for testing rules safely
- **-Q**
  inline mode, two interfaces bridged
- **-k none**
  skip checksum checks on a mirror port

Zeek: How It Works



- Formerly Bro (Vern Paxson, 1990s), renamed Zeek in 2018
- A Network Security Monitor, not mainly signature-based
- Passive: watches traffic and writes detailed logs, has its own scripting language
- Used in Security Onion, research networks and enterprise SOCs
- **Like a court stenographer**
  records who connected to whom, which DNS names were asked and which certificates were used

Zeek: The Logs

Log	Contents	Example use
`conn.log`	Every connection: IPs, ports, duration, bytes	Exfiltration, beaconing
`dns.log`	DNS queries and answers	Malware domains, DNS tunneling
`http.log`	Requests, user-agent, URI	Suspicious web activity
`ssl.log`	TLS version, certificates, SNI	Self-signed or expired certs
`files.log`	Files seen, with hashes	Malware hash lookup
`notice.log`	Events flagged as suspicious	Alert triage

Zeek: Deployment

ZEEKCTL

```
# /opt/zeek/etc/node.cfg
[zeek]
type=standalone
host=localhost
interface=eth0

# /opt/zeek/etc/networks.cfg
10.0.0.0/8        Private
192.168.0.0/16    Private

# local.zeek: JSON logs for SIEM
@load policy/tuning/json-logs

sudo /opt/zeek/bin/zeekctl deploy
sudo /opt/zeek/bin/zeekctl status
```

- **standalone**
  one machine does everything, fine for labs
- **interface**
  should be a SPAN port or network TAP
- **Logs**
  in /opt/zeek/logs/current/, rotated into dated folders
- Paths depend on how Zeek was installed

Zeek: Analyzing Logs

ZEEK-CUT

```
cd /opt/zeek/logs/current

# Biggest uploads
cat conn.log | zeek-cut id.orig_h id.resp_h orig_bytes \
  | sort -k3 -nr | head

# Top talkers
cat conn.log | zeek-cut id.orig_h \
  | sort | uniq -c | sort -nr | head

# Most asked DNS names
cat dns.log | zeek-cut query | sort | uniq -c | sort -nr | head

# Replay a pcap
zeek -r capture.pcap
```

> *With JSON logs enabled, use jq instead of zeek-cut. Replaying a pcap writes fresh logs into the current folder.*

Zeek: Scripts and Threat Hunting

SCRIPT

```
# long-dns.zeek
event dns_request(c: connection, msg: dns_msg,
                  query: string, qtype: count, qclass: count)
{
  if ( |query| > 50 )
    print fmt("Long DNS from %s: %s",
              c$id$orig_h, query);
}

# Run it on a pcap
zeek -r capture.pcap long-dns.zeek
```

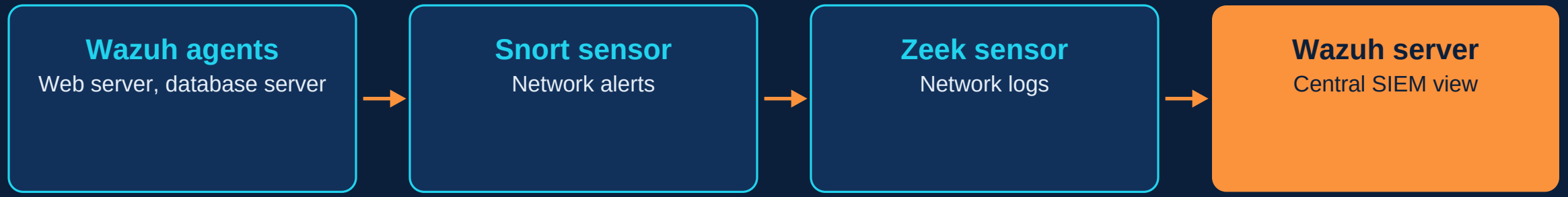
- **Beaconing**
  regular check-ins in conn.log suggest malware C2
- **DNS tunneling**
  very long domain names
- **Retrospective search**
  when a new IOC appears, search old logs for it

Comparison: Wazuh vs Snort vs Zeek

	Wazuh	Snort	Zeek
Scope	Host	Network	Network
Method	Logs, FIM, rules	Signatures	Metadata + scripts
Can block?	Yes (Active Response)	Yes (IPS mode)	No (passive)
Output	Alerts + dashboard	Alerts	Rich logs
Best at	Endpoint, compliance	Known attacks, blocking	Forensics, hunting

> *Snort = guard at the gate. Zeek = CCTV with recording. Wazuh = guard inside each room.*

Layered Setup: Better Together



- Network + host correlation: scan seen by Snort + connection in Zeek + failed SSH logins in Wazuh = one clear attack story
- Put a Wazuh agent on each sensor so Snort alerts and Zeek logs reach the same dashboard
- One place to search, one place to alert

Alert Triage Workflow

- **Zeek conn.log**
  shows everything the suspect host talked to
- **Wazuh**
  shows what changed on the host: logins, files, processes
- Every false positive is a tuning task, not just noise

RECAP

Summary and Recommendations

- **Wazuh**
  host guard: logs, FIM, vulnerabilities, active response
- **Snort**
  network guard: signatures, optional blocking
- **Zeek**
  network recorder: rich logs, hunting
- Start with Wazuh (easiest to deploy), then add Snort or Zeek
- Always tune rules to reduce false positives

Q & A

Detect. Analyze. Respond.